PRODUCTION READINESS GUIDE

Composable Agent Stack

Verification & AI Prompts

A systematic framework for validating production readiness and extracting deep architectural knowledge across Agent-S, Browser Use, OpenHands, and LiteLLM

`Agent-S` `Browser Use` `OpenHands` `LiteLLM Router`

> Covers six verification dimensions: pre-integration testing, integration validation, security auditing, performance benchmarking, license compliance, and production deployment. Includes 14 ready-to-use AI prompts for deep knowledge extraction across all platforms.

Author: Z.ai

Date: May 2026

Stack: Agent-S (Apache-2.0) + Browser Use (MIT) + OpenHands (MIT) + LiteLLM (MIT)

Table of Contents

1. Executive Summary

This document provides a comprehensive production-readiness verification framework and a set of AI-powered knowledge extraction prompts for the composable agent stack consisting of Agent-S (desktop automation, Apache-2.0), Browser Use (web automation, MIT), OpenHands (software development platform, MIT), and LiteLLM (unified LLM router, MIT). The stack was selected for maximum flexibility with minimum vendor lock-in, enabling you to swap any component independently while maintaining a cohesive orchestration layer through LiteLLM.

The verification framework covers six critical dimensions: pre-integration validation of each component, integration testing across the composable stack, security auditing, performance and reliability benchmarking, license and compliance verification, and a production deployment readiness checklist. Each dimension includes specific test cases, acceptance criteria, and remediation steps. The AI prompts section provides carefully crafted queries you can paste into any large language model (ChatGPT, Claude, Gemini, DeepSeek, etc.) to extract deep architectural knowledge, troubleshooting workflows, integration patterns, and step-by-step implementation guidance for each platform.

The core principle underlying this entire document is that production readiness is not a single gate but a continuous spectrum. Each verification dimension must pass independently before the stack can be considered production-grade. A failure in any single dimension (for example, a security vulnerability in Browser Use or a performance bottleneck in LiteLLM routing) must be resolved before proceeding to deployment. This document gives you the tools, checklists, and AI-driven knowledge extraction methods to systematically close every gap.

2. Pre-Integration Verification

Before composing the stack, each component must independently pass a set of baseline verification tests. This ensures that when integration issues arise, you can confidently isolate them to the integration layer rather than a defective component. The following subsections detail the verification steps for each of the four platforms, covering functional correctness, dependency health, configuration flexibility, and known issue identification.

2.1 Agent-S (Desktop Automation)

Agent-S provides full desktop control through screenshot-based interaction, enabling AI agents to click, type, scroll, and navigate any application on Windows, macOS, or Linux. It uses a visual grounding approach that translates natural language instructions into precise mouse and keyboard actions. Before integrating Agent-S into your composable stack, verify the following dimensions to ensure it functions correctly in isolation.

Functional Verification

- Clone the repository and run the official quickstart example without modification. The agent should successfully launch a desktop application (e.g., Notepad on Windows, TextEdit on macOS) and type a predefined string. Record the success rate across 5 consecutive runs; a passing result requires at least 4 out of 5 successful completions.
- Test multi-step task execution: instruct the agent to open a file manager, navigate to a specific directory, create a new folder, and rename it. Each sub-task must complete within 30 seconds, and the overall task chain must complete within 120 seconds.
- Verify screenshot capture fidelity: the agent must capture screenshots at the native display resolution without scaling artifacts. Compare the captured screenshot dimensions against your actual display resolution. Any mismatch indicates a rendering pipeline issue that will degrade visual grounding accuracy.
- Test error recovery: deliberately introduce a scenario where the agent clicks a non-existent UI element. Verify that the agent detects the failure (no state change after action) and retries with an alternative strategy rather than entering an infinite loop.

Dependency and Build Health

- Run pip install -e . in a fresh virtual environment (Python 3.10+). Verify zero dependency conflicts. If any dependency resolution errors occur, document them and check the project issue tracker for known workarounds.
- Run the full test suite (pytest tests/). Record the number of passing, failing, and skipped tests. A production candidate should have less than 5% failing tests with no critical-path failures (core actions like click, type, scroll).
- Check for pinned vs. unpinned dependencies in pyproject.toml or requirements.txt. Unpinned dependencies (using >= without upper bounds) risk silent breakage on future updates. Prefer versions with upper-bounded pins or lockfiles.

LLM Provider Flexibility

- Configure Agent-S to use at least three different LLM providers: OpenAI GPT-4o, Anthropic Claude, and a local model via Ollama. Each provider must successfully complete the same desktop task. This validates that the agent is not subtly hard-coded to a specific provider response format.
- Verify that switching providers requires only configuration changes (environment variables or config file edits) with zero code modifications. If any code changes are needed, the abstraction layer is leaking and must be refactored before integration.

2.2 Browser Use (Web Automation)

Browser Use enables AI agents to interact with web pages through structured DOM extraction, intelligent element detection, and multi-tab management. Unlike traditional Selenium or Playwright scripts that require explicit selectors, Browser Use lets the agent identify elements by natural language description. It supports 15+ LLM providers out of the box, making it the most vendor-flexible

component in the stack. Verify the following before integration.

Functional Verification

- Run the basic example: navigate to Google, search for a query, and extract the first result title. This tests the core loop of navigation, interaction, and data extraction. Run 5 times; at least 4 must succeed.
- Test multi-tab workflow: instruct the agent to open two tabs, perform different tasks in each, then merge results. This validates tab lifecycle management and context isolation.
- Verify form interaction: fill out a multi-field form (at least 5 fields including text input, dropdowns, checkboxes, and radio buttons) and submit. The agent should correctly identify and interact with each element type.
- Test dynamic content handling: navigate to a single-page application (e.g., a React dashboard) and verify the agent can interact with dynamically loaded content that appears after scrolling or clicking. This tests the agent ability to wait for content loading and re-extract the DOM after state changes.
- Test cookie/session persistence: log into a website, close the browser session, reopen it with the same profile, and verify the session persists. This is critical for workflows that span multiple automated sessions.

LLM Provider Matrix

Browser Use claims support for 15+ LLM providers. Validate this by configuring at least the following providers and running the same web automation task with each:

Provider	Configuration Method	Expected Latency	Validation Status
OpenAI (GPT-4o)	OPENAI_API_KEY env var	2-5s per step	Pending
Anthropic (Claude)	ANTHROPIC_API_KEY env var	3-6s per step	Pending
Google (Gemini)	GEMINI_API_KEY env var	2-4s per step	Pending
Ollama (Local)	OLLAMA_BASE_URL env var	1-3s per step	Pending
LiteLLM (Router)	LITELLM_BASE_URL env var	Varies	Pending

Anti-Detection and Stealth

- Run Browser Use against a bot-detection service (e.g., bot.sannysoft.com or pixelscan.net). Verify that the browser fingerprint appears as a normal user browser, not an automated instance. Key indicators: navigator.webdriver must be false, Chrome DevTools detection must be negative, and canvas/WebGL fingerprints must be consistent with a real browser profile.

- Test against a Cloudflare-protected website. The agent must successfully pass the Cloudflare challenge without manual intervention. If it fails, this is a known limitation that must be documented and potentially addressed with additional stealth plugins.

2.3 OpenHands (Software Development Platform)

OpenHands is a full-featured AI-powered software development platform with a web UI, sandboxed code execution, git integration, and multi-agent collaboration. Unlike Agent-S and Browser Use which focus on runtime interaction, OpenHands excels at development-time tasks: writing code, debugging, running tests, and managing pull requests. It runs each agent session in an isolated Docker sandbox, providing strong security boundaries.

Functional Verification

- Launch OpenHands via Docker Compose. Verify the web UI loads at localhost:3000 and the backend API responds at localhost:8000. Both must be accessible without TLS in development mode.
- Create a new workspace and instruct the agent to scaffold a basic Python project (create a virtual environment, install pytest, write a hello.py with a failing test, run the test, and fix the code). The agent must complete all steps without human intervention.
- Test git integration: instruct the agent to initialize a git repo, make a commit, create a branch, and open a pull request (if connected to a GitHub/GitLab instance). Each git operation must succeed and be reflected in the repository.
- Verify sandbox isolation: run a destructive command (e.g., rm -rf /tmp/test_sandbox) inside the agent sandbox. Confirm that the host filesystem is unaffected. The sandbox must not have access to host resources beyond the explicitly mounted workspace directory.
- Test multi-agent collaboration: launch two agents in the same workspace with different tasks (e.g., one writes a function, the other writes tests for it). Verify they can work concurrently without file conflicts using proper locking or sequential coordination.

Configuration Flexibility

- Verify that the LLM provider can be switched by changing a single environment variable (LLM_MODEL and LLM_API_KEY). Test with at least three providers: OpenAI, Anthropic, and a local model.
- Verify that the sandbox runtime can be configured (Docker vs. local vs. Kubernetes). At minimum, Docker mode must work out of the box.
- Test the headless CLI mode: run OpenHands without the web UI, providing instructions via command line. This validates that the platform can be embedded in CI/CD pipelines and automated workflows.

2.4 LiteLLM (LLM Router)

LiteLLM serves as the universal LLM router for the entire composable stack, providing a unified OpenAI-compatible API that translates requests to any of 100+ LLM providers. It handles authentication, rate limiting, fallback routing, cost tracking, and model aliasing. As the single point of LLM configuration, LiteLLM must be rock-solid before integration. A failure in LiteLLM propagates to all three agent platforms simultaneously.

Functional Verification

- Install LiteLLM and start the proxy server with a minimal config.yaml containing at least two providers (e.g., OpenAI and Anthropic). Verify the server starts without errors and responds to health check requests at /health.
- Send a chat completion request through the proxy to each configured provider. Verify the response format is identical regardless of the underlying provider (OpenAI-compatible schema with choices, message, usage fields).
- Test fallback routing: configure provider A as primary and provider B as fallback. Simulate a failure of provider A (invalid API key or rate limit) and verify that LiteLLM automatically routes the request to provider B without returning an error to the client.
- Test rate limiting: configure a low rate limit (e.g., 5 requests per minute) and send 10 requests rapidly. Verify that the first 5 succeed and the remaining 5 are either queued or return a 429 status code with a Retry-After header.
- Test cost tracking: make 10 requests with known token counts, then query the spend endpoint. Verify the reported cost matches the expected cost based on the configured per-token pricing for each model.

Performance Benchmarks

LiteLLM adds a thin routing layer between the client and the LLM provider. This layer must introduce minimal latency. Run the following benchmarks:

Metric	Target	Measurement Method
Proxy overhead (p50)	< 50ms	Compare direct API call vs. proxied call latency
Proxy overhead (p99)	< 200ms	Same as above, 99th percentile
Throughput (req/s)	> 100 req/s	Sustained load test with concurrent requests
Memory usage (idle)	< 200MB	RSS memory of proxy process with no active requests
Memory usage (load)	< 1GB	RSS memory under 100 concurrent requests

3. Integration Verification

Once each component passes pre-integration verification independently, the next phase validates that they work together as a composable stack. Integration testing focuses on the seams between components: how Agent-S, Browser Use, and OpenHands communicate through LiteLLM, how state is shared or isolated across platforms, and how failures in one component affect the others. The goal is to ensure the stack is truly composable, meaning each component can be started, stopped, and replaced independently without cascading failures.

3.1 LiteLLM as Unified Router

The first integration test validates that all three agent platforms route their LLM calls through LiteLLM rather than making direct API calls. This is the foundational integration point. If any platform bypasses LiteLLM, you lose centralized cost tracking, rate limiting, and provider switching capability.

Agent-S + LiteLLM

- Configure Agent-S to use LiteLLM as its LLM backend by setting the API base URL to the LiteLLM proxy endpoint (e.g., http://localhost:4000). Verify that all LLM requests from Agent-S appear in the LiteLLM logs with the correct model name and token usage.
- Test provider switching: change the model in the LiteLLM config from gpt-4o to claude-sonnet-4-20250514 without modifying any Agent-S configuration. Run the same desktop task. The task must succeed with both models, confirming that provider switching is handled entirely by LiteLLM.

Browser Use + LiteLLM

- Configure Browser Use to point to the LiteLLM proxy. Run a web automation task and verify the request appears in LiteLLM logs.
- Test the fallback scenario: temporarily disable the primary provider in LiteLLM config. Browser Use must continue operating via the fallback provider with no code changes required on the Browser Use side.

OpenHands + LiteLLM

- Set LLM_BASE_URL to the LiteLLM proxy endpoint in the OpenHands Docker environment. Instruct the agent to write a Python function and run tests. Verify that all LLM requests (including any internal tool-calling steps) are routed through LiteLLM.
- Verify that OpenHands streaming responses work correctly through the LiteLLM proxy. Some proxy implementations buffer streaming responses, which breaks the real-time UI updates in OpenHands. Test by observing the web UI during code generation; tokens should appear incrementally, not all at once.

3.2 Cross-Platform Orchestration

Cross-platform orchestration tests validate that the three agent platforms can be coordinated to work on a shared workflow. Unlike single-platform tests, these tests exercise the communication and state-sharing mechanisms between platforms. The orchestration layer (which you will build) must be able to dispatch tasks to the appropriate platform, collect results, and trigger follow-up actions on other platforms.

End-to-End Workflow Test

Design a workflow that exercises all three platforms in sequence:

- Step 1 (OpenHands): Agent generates a Python web scraper for a target website.
- Step 2 (Browser Use): Agent deploys the scraper and validates it works against the live website, handling any CAPTCHA or login requirements.
- Step 3 (Agent-S): Agent opens a desktop spreadsheet application, imports the scraped data, and applies formatting.

Each step must produce a verifiable output artifact (a .py file, a .csv file, and a .xlsx file respectively). The orchestrator must pass the output of each step as input to the next step. A successful end-to-end completion validates the entire communication pipeline.

Failure Isolation Test

- Simulate a failure in Browser Use (e.g., kill the Browser Use process mid-task). Verify that Agent-S and OpenHands continue operating independently. The orchestrator must detect the Browser Use failure and either retry the task or route it to an alternative platform, without crashing the entire stack.
- Simulate a LiteLLM outage (stop the proxy server). All three platforms should fail gracefully with clear error messages indicating the LLM router is unavailable, rather than hanging indefinitely or producing cryptic authentication errors.

3.3 State Management and Context Passing

When agents across different platforms collaborate, they need to share context. This context includes task descriptions, intermediate results, file paths, authentication tokens, and error states. The verification must ensure that context is passed reliably and securely between platforms without data loss or corruption.

- File-based context: Verify that files created by one platform (e.g., a Python script from OpenHands) are accessible by another platform (e.g., Browser Use running the script). Test with files containing Unicode characters, long paths (over 200 characters), and binary data.
- API-based context: If the orchestrator uses an internal API to pass context between platforms, test the API with payloads of varying sizes (100 bytes to 10 MB). Verify that large payloads do not cause timeouts or memory errors.

- Authentication context: If Browser Use obtains a session cookie from a website, verify that this cookie can be securely stored and retrieved by Agent-S for subsequent desktop-based interactions with the same service. Sensitive authentication data must never be logged in plaintext.

4. Security Audit

Security auditing for an AI agent stack is fundamentally different from traditional application security. These agents execute arbitrary actions on behalf of users, including clicking, typing, navigating, and running code. A compromised agent can cause real-world damage: deleting files, making unauthorized purchases, or exfiltrating sensitive data. The security audit must cover each component individually and the stack as a whole, with particular attention to the expanded attack surface created by composition.

4.1 Per-Component Security

Agent-S Security

- Input validation: Test the agent with malicious instructions (e.g., "delete all files in the home directory", "send all passwords to example.com"). The agent must either refuse or prompt for explicit user confirmation before executing destructive actions. Verify that there is a confirmation gate for high-risk operations.
- Screenshot privacy: Agent-S captures screenshots of the entire desktop. Verify that sensitive applications (password managers, banking sites) can be excluded from screenshot capture via a configurable exclusion list. Check that screenshots are not persisted to disk after the agent session ends.
- Privilege isolation: Verify that the agent runs with the minimum necessary OS permissions. It must not require root/admin access for normal operation. Test by running the agent under a restricted user account and verifying all basic tasks still function.

Browser Use Security

- Credential exposure: When the agent logs into a website, verify that API keys, passwords, and session tokens are not logged in the agent output or stored in plaintext configuration files. Check the logs directory for any credential leakage.
- Cross-site context leakage: If the agent operates across multiple tabs simultaneously, verify that cookies and localStorage from one tab are not accessible to scripts in another tab. This is typically enforced by the browser same-origin policy, but custom browser configurations may weaken it.
- Download safety: If the agent downloads files, verify they are stored in a sandboxed directory with no execute permissions. Downloaded executables must not be automatically run.

OpenHands Security

- Sandbox escape: Attempt to break out of the Docker sandbox using known container escape techniques (e.g., accessing /proc/1/root, mounting the Docker socket, exploiting cgroup vulnerabilities). The sandbox must contain all such attempts.
- Network isolation: Verify that the sandbox cannot access internal network services that are not explicitly allowed. Test by attempting to connect to a test HTTP server running on the host network. The connection must be blocked unless the service is explicitly exposed to the sandbox.
- Secrets management: Verify that API keys passed to the agent are not embedded in generated code or committed to git repositories. The platform should provide a secrets injection mechanism that keeps credentials out of the workspace volume.

LiteLLM Security

- API key storage: Verify that LLM provider API keys stored in the LiteLLM config are encrypted at rest, not stored as plaintext in config.yaml. If using environment variables, verify they are not exposed in process listings or log output.
- Request logging: Check what LiteLLM logs by default. Prompt content and model responses may contain sensitive data. Verify that there is a configuration option to disable prompt/response logging while retaining metadata logging (model, token count, latency).
- Access control: If exposing LiteLLM as a network service, verify that it supports API key authentication for clients. Without this, any network client can consume your LLM credits.

4.2 Stack-Level Security

The composable architecture introduces security considerations that do not exist in any single component alone. When agents from different platforms share files, communicate through APIs, and coordinate through a central orchestrator, new attack vectors emerge that must be explicitly tested and mitigated.

- Lateral movement prevention: If one agent platform is compromised, verify that it cannot directly control or manipulate another platform. There must be no shared authentication tokens or direct API access between agent platforms. All communication must go through the orchestrator, which enforces access control policies.
- Audit trail integrity: Verify that all agent actions across all platforms are logged in a tamper-evident audit log. The log must record the timestamp, agent identity, action type, and target resource for every operation. The audit log must be append-only and not modifiable by the agents themselves.
- Rate limiting across the stack: A compromised agent could consume LLM credits at an accelerated rate. Verify that LiteLLM rate limits apply across all connected platforms collectively, not per-platform. A single rogue agent must not be able to exhaust the entire rate limit budget.

5. Performance and Reliability Testing

Performance and reliability testing ensures the composable stack can handle real-world workloads at scale. An AI agent stack is unique in that its performance is dominated by LLM inference latency (typically 2-10 seconds per step) rather than traditional CPU or memory constraints. However, the orchestration layer, file I/O, browser rendering, and desktop interaction all contribute to end-to-end latency. This section defines benchmarks and acceptance criteria for both steady-state and adversarial conditions.

5.1 Latency Benchmarks

Measure end-to-end latency for representative tasks across each platform, both in isolation and as part of the integrated stack. The following table defines target latencies for common operations. These targets assume GPT-4o-class model performance; local models will be slower proportionally.

Operation	Standalone Target	Integrated Target	Max Acceptable
Agent-S: single click action	3-6s	4-8s	15s
Agent-S: multi-step task (5 steps)	15-30s	20-40s	90s
Browser Use: navigate + extract	5-10s	6-12s	20s
Browser Use: form fill (5 fields)	15-25s	18-30s	60s
OpenHands: write + test a function	30-60s	35-70s	120s
OpenHands: full project scaffold	2-5 min	2.5-6 min	15 min
LiteLLM: routing overhead (p50)	N/A	< 50ms	200ms
Cross-platform workflow (3 steps)	N/A	1-3 min	10 min

5.2 Reliability Testing

Long-Running Sessions

AI agent sessions can run for minutes to hours. Memory leaks, connection timeouts, and state drift can cause failures that only manifest after extended operation. Run the following long-duration tests:

- Agent-S 1-hour session: Run a continuous sequence of desktop tasks for one hour. Monitor memory usage (RSS) every 5 minutes. Memory growth must not exceed 50MB over the baseline. If memory grows linearly, there is a leak that must be fixed before production deployment.
- Browser Use 100-page crawl: Instruct the agent to visit 100 distinct web pages and extract data from each. Verify that the browser process does not crash, memory does not grow unbounded, and the agent maintains context across all 100 pages.
- OpenHands multi-day workspace: Create a workspace and use it intermittently over 3 days (opening and closing sessions). Verify that the workspace state is correctly preserved between sessions and that the Docker sandbox starts cleanly each time.

Network Resilience

- LLM provider outage: Simulate a complete outage of the primary LLM provider by setting an invalid API key. Verify that LiteLLM falls back to the configured backup provider within 10 seconds, and that in-flight requests are either retried on the backup or returned with a clear error.
- Intermittent network: Introduce 10% packet loss on the network interface. Verify that the agent platforms handle request failures gracefully with exponential backoff, rather than immediately failing the entire task.
- DNS resolution failure: Temporarily break DNS resolution for the LLM provider domain. Verify that the error message clearly indicates a DNS failure, not a cryptic connection timeout or authentication error.

5.3 Concurrent Operation

In production, you may need to run multiple agent instances simultaneously (e.g., three Browser Use agents scraping different websites, or two OpenHands agents working on different codebases). Verify the following concurrent operation scenarios:

- Concurrent Agent-S instances: Launch two Agent-S instances on the same desktop, each targeting different applications. Verify that screen capture correctly isolates the target application window and does not confuse the two agents. If both agents capture the full screen, they will see each other actions, leading to unpredictable behavior.
- Concurrent LiteLLM requests: Send 50 simultaneous requests through LiteLLM to the same provider. Verify that all 50 requests complete successfully (or are correctly queued if the provider rate limit is hit). No requests should be dropped silently.
- Concurrent file access: Have OpenHands and Browser Use simultaneously write to the same directory. Verify that no file corruption occurs. If both agents write to the same file, the last writer should win (not a interleaved corruption).

6. License and Compliance Verification

All four components of the composable stack use permissive open-source licenses, but the specific terms differ and must be carefully verified, especially if you plan commercial use. License compliance is not merely a legal checkbox; it affects how you can distribute, modify, and patent your derived work. This section provides a detailed license analysis and a compliance checklist.

Component	License	Commercial Use	Modification	Distribution	Patent Grant	Copyleft
Agent-S	Apache-2.0	Yes	Yes	Yes	Yes	No
Browser Use	MIT	Yes	Yes	Yes	No	No
OpenHands	MIT	Yes	Yes	Yes	No	No
LiteLLM	MIT	Yes	Yes	Yes	No	No

6.1 Compliance Checklist

- Attribution notices: Apache-2.0 requires that you include the original copyright notice and license text in any distributed copy. For Agent-S, create a NOTICES file in your project that includes the Agent-S copyright and Apache-2.0 license text. MIT licenses require including the copyright notice and license text in substantial copies of the software.
- State changes notice: Apache-2.0 requires that you state significant changes made to the original files. If you modify Agent-S source code, add a prominent notice in the modified files indicating what was changed and when.
- Trademark usage: None of the four licenses grant trademark rights. You cannot use the names " Agent-S", "Browser Use", "OpenHands", or "LiteLLM" to endorse or promote your product without explicit written permission from the respective trademark holders.
- Patent retaliation clause: Apache-2.0 includes a patent retaliation clause (Section 3). If you initiate patent litigation against any contributor alleging that the software constitutes patent infringement, your patent license from that contributor terminates automatically. MIT has no such clause, meaning you have no explicit patent protection from Browser Use, OpenHands, or LiteLLM contributors.
- Dependency licenses: Each component has its own dependency tree with potentially different licenses. Run a license scan using a tool like FOSSA, Snyk, or pip-licenses to verify that no transitive dependency introduces a restrictive license (GPL, AGPL, SSPL) that could contaminate your project.

7. Production Deployment Checklist

The following checklist must be completed before the composable agent stack is deployed to a production environment. Each item has a designated owner and a verification method. No item should be marked complete based on assumption; every item requires explicit evidence (test results, configuration screenshots, audit logs, or peer review sign-off).

Checklist Item	Owner	Verification Method
All pre-integration tests pass	QA	Test results in CI dashboard
All integration tests pass	QA	End-to-end workflow completion
Security audit completed with no critical findings	Security	Audit report with findings and remediation
Rate limiting configured in LiteLLM	DevOps	Load test showing 429 responses at limit
Fallback provider configured and tested	DevOps	Failover test with primary provider disabled
API keys stored in secrets manager (not env vars on disk)	DevOps	Secrets manager audit log
Audit logging enabled across all platforms	Security	Log sample showing all action types
Memory leak tests pass (1-hour run)	QA	Memory graph showing flat RSS
Network resilience tests pass	QA	Test results with simulated outages
Concurrent operation tests pass	QA	Test results with 3+ concurrent agents
License compliance verified (all dependencies)	Legal	FOSSA or Snyk license scan report
Attribution notices included in distribution	Legal	NOTICES file review
Backup and disaster recovery plan documented	DevOps	DR playbook with recovery time objective
Monitoring and alerting configured	DevOps	Alert triggered during load test
Documentation for operators and developers	Engineering	Docs review sign-off
Incident response playbook for agent failures	Engineering	Playbook review sign-off

8. AI Prompts for Knowledge Extraction

The following prompts are designed to be pasted directly into any capable large language model (ChatGPT, Claude, Gemini, DeepSeek, etc.) to extract deep architectural knowledge, integration patterns, and step-by-step implementation guidance for each component of the composable agent stack.

Each prompt is self-contained with sufficient context to produce a high-quality response even without prior conversation history. The prompts are organized by platform and then by knowledge domain (architecture, integration, troubleshooting, optimization).

> How to use these prompts: Copy the prompt text verbatim into your preferred AI assistant. You may add specific context (your OS, your Python version, your use case) at the end of the prompt for more targeted responses. For best results, start a new conversation for each prompt to avoid context contamination from unrelated discussions.

8.1 Agent-S Prompts

PROMPT: Architecture Deep Dive

I am building a composable AI agent stack that combines Agent-S for desktop automation, Browser Use for web automation, OpenHands for software development, and LiteLLM as the unified LLM router. I need a deep architectural understanding of Agent-S. Please provide: (1) A detailed explanation of the Agent-S visual grounding pipeline, from screenshot capture to action execution, including how it identifies UI elements without DOM access. (2) The internal state machine that governs the agent decision loop: how does it decide when to take an action vs. when to wait for a UI change vs. when to report task completion? (3) The exact mechanism for switching LLM providers: what interface does Agent-S use to call LLMs, and what changes are needed to route all calls through a LiteLLM proxy at http://localhost:4000? (4) Known limitations: what types of desktop applications or UI frameworks does Agent-S struggle with (e.g., Electron apps, Java Swing, remote desktop sessions)? (5) The security model: what OS permissions does Agent-S require, and how can I restrict its capabilities to prevent accidental damage?

PROMPT: Integration with LiteLLM Proxy

I am integrating Agent-S with LiteLLM as a unified LLM router. LiteLLM is running as a proxy server at http://localhost:4000 with a config.yaml that defines model aliases (e.g., "smart" maps to gpt-4o, "fast" maps to gpt-4o-mini, "local" maps to ollama/llama3). Please provide: (1) The exact configuration changes needed in Agent-S to point all LLM calls to the LiteLLM proxy instead of direct API calls. Include the specific environment variables or config file modifications. (2) How to handle streaming responses: does Agent-S use streaming, and if so, does LiteLLM proxy streaming correctly? (3) How to implement fallback routing so that if the "smart" model fails, Agent-S automatically falls back to the "fast" model via LiteLLM configuration, with zero code changes in Agent-S. (4) A testing script that sends 10 requests through the proxy and verifies each returns a valid response with correct model attribution in the response metadata. (5) Common pitfalls when using LiteLLM with screenshot-heavy agents: are there token limits, image encoding issues, or latency concerns I should be aware of?

PROMPT: Production Deployment Checklist

I am deploying Agent-S to a production environment where it will run unattended desktop automation tasks on Windows 11 workstations. Please provide: (1) A step-by-step deployment guide including Python environment setup, dependency installation, and agent configuration. (2) How to run Agent-S as a background service that starts automatically on boot and restarts on crash. (3) Monitoring strategy: what metrics should I track (success rate, latency, memory usage, error rate) and how to expose them via

Prometheus or a similar monitoring system. (4) How to implement a dead man switch that alerts me if the agent has not completed a task within a configurable timeout period. (5) How to securely store and rotate the API keys used by Agent-S without restarting the agent process. (6) A rollback strategy: if an agent update introduces a regression, how can I quickly revert to the previous working version while minimizing downtime?

8.2 Browser Use Prompts

PROMPT: Architecture Deep Dive

I am evaluating Browser Use for production web automation within a composable AI agent stack (alongside Agent-S for desktop, OpenHands for code, and LiteLLM as the LLM router). Please provide: (1) A detailed explanation of how Browser Use extracts and represents the DOM for LLM consumption. How does it handle dynamic content (SPAs, infinite scroll, lazy loading)? (2) The element detection and interaction pipeline: how does it map a natural language instruction like "click the submit button" to a specific DOM element? What happens when multiple elements match the description? (3) Multi-tab and multi-window management: how does Browser Use maintain separate contexts for different tabs, and can an agent operate on two tabs simultaneously? (4) The exact interface for LLM integration: what API format does Browser Use expect from the LLM, and how can I route all calls through a LiteLLM proxy? (5) Anti-detection mechanisms: what built-in features does Browser Use have to avoid bot detection (fingerprint randomization, human-like delays, etc.), and how do they compare to dedicated stealth tools like undetected-chromedriver?

PROMPT: Vendor-Agnostic LLM Configuration

I want to make Browser Use fully vendor-agnostic by routing all LLM calls through LiteLLM (running at http://localhost:4000). The LiteLLM config has aliases: "smart" for GPT-4o, "fast" for GPT-4o-mini, "local" for Ollama Llama3, and "claude" for Claude Sonnet. Please provide: (1) The exact code or configuration changes needed to route Browser Use LLM calls through LiteLLM. I need to see the specific file paths, variable names, and values. (2) How to implement a dynamic model selection strategy where the orchestrator chooses "smart" for complex navigation tasks and "fast" for simple data extraction, all through LiteLLM model aliases. (3) A test script that runs the same web task (e.g., search Google and extract the first result) with each of the 4 model aliases and compares success rate and latency. (4) How to handle Browser Use token limits when using models with smaller context windows (e.g., local models with 4K context). What is the typical token consumption for a single browser step, and how can I optimize it? (5) Fallback strategy: if the "smart" model returns an invalid action (e.g., clicking a non-existent element), how can I configure LiteLLM to retry with the "fast" model automatically?

PROMPT: Scaling and Reliability

I need to run 10 concurrent Browser Use agent sessions, each on a different website, all routing through a single LiteLLM proxy. Please provide: (1) Resource requirements: how much RAM and CPU does each Browser Use session consume? What is the minimum hardware specification for 10 concurrent sessions? (2) Browser instance management: does each session need a separate browser instance, or can they share a browser with different profiles? How to configure this? (3) Rate limiting strategy: with 10 agents each making LLM calls every 5-10 seconds, I am consuming 1-2 requests per second. How should I configure LiteLLM rate limits to stay within provider quotas (e.g., OpenAI 500 RPM for GPT-4o)? (4) Session

persistence: if a Browser Use session crashes, how can I resume from the last completed step rather than restarting the entire task? Does Browser Use support checkpointing? (5) Monitoring: what are the key health indicators for a Browser Use session, and how can I expose them to a monitoring system (Prometheus, Datadog, etc.)?

8.3 OpenHands Prompts

PROMPT: Architecture Deep Dive

I am integrating OpenHands into a composable AI agent stack that also includes Agent-S (desktop automation), Browser Use (web automation), and LiteLLM (unified LLM router). Please provide: (1) A detailed explanation of the OpenHands runtime architecture: how the web UI, backend API, and Docker sandbox interact. What are the communication protocols between these components? (2) The sandbox execution model: how does OpenHands execute code inside the Docker container, and what security boundaries prevent sandbox escape? Can I customize the sandbox image (e.g., add Node.js, Go, or Rust toolchains)? (3) The agent loop: how does OpenHands decide when to write code, when to run tests, when to search the codebase, and when to ask the user for clarification? What is the prompt structure sent to the LLM? (4) Multi-agent capabilities: how does OpenHands support multiple agents working on the same codebase? Is there built-in coordination, or do I need to build my own orchestration? (5) The exact mechanism for LLM provider configuration: what environment variables control the LLM endpoint, and how can I route all calls through a LiteLLM proxy at http://localhost:4000?

PROMPT: LiteLLM Integration and Custom Sandbox

I am configuring OpenHands to route all LLM calls through LiteLLM (proxy at http://localhost:4000) and to use a custom Docker sandbox image with Python 3.12, Node.js 20, and Go 1.22 pre-installed. Please provide: (1) The exact Docker Compose modifications needed to set LLM_BASE_URL, LLM_API_KEY, and LLM_MODEL environment variables to point to LiteLLM with a model alias like "openhands-smart". (2) How to build a custom sandbox Docker image: the Dockerfile structure, what base image to extend, and how to register it with OpenHands. (3) How to verify that all LLM requests from OpenHands are actually going through LiteLLM. I need a way to confirm no requests are bypassing the proxy. (4) Streaming response validation: how to verify that token streaming works correctly through LiteLLM, so the OpenHands web UI shows incremental output. (5) How to configure different models for different task types within OpenHands (e.g., a faster model for simple file edits, a smarter model for debugging), using LiteLLM model aliases.

PROMPT: CI/CD and Headless Operation

I want to use OpenHands in a CI/CD pipeline where it receives a task description as a command-line argument, executes it in a sandbox, and outputs the result. No web UI needed. Please provide: (1) The exact command to run OpenHands in headless/CLI mode with a task description. Include all required flags and environment variables. (2) How to capture the output: what exit codes does OpenHands use, and where does it write the generated code and test results? (3) How to integrate with GitHub Actions: a complete workflow YAML file that runs OpenHands on a pull request, generates code, runs tests, and posts the results as a PR comment. (4) Timeout handling: how to set a maximum execution time for a task so that runaway agents are killed automatically. (5) How to pass context to the agent (e.g., the PR diff, the issue description, relevant documentation) via the CLI interface rather than through the web UI.

8.4 LiteLLM Prompts

PROMPT: Complete Configuration for Agent Stack

I am setting up LiteLLM as the unified LLM router for a composable agent stack consisting of Agent-S (desktop automation), Browser Use (web automation), and OpenHands (software development). I need a complete config.yaml that supports the following requirements: (1) Four model aliases: "smart" for GPT-4o, "fast" for GPT-4o-mini, "local" for Ollama Llama3, and "claude" for Claude Sonnet 4. (2) Fallback routing: if "smart" fails (rate limit, timeout, or 5xx error), fall back to "claude", then "fast", then "local". (3) Rate limiting: 60 requests per minute for "smart", 120 for "fast", 30 for "local", 60 for "claude". (4) Cost tracking: enable spend tracking for all models with realistic pricing. (5) API key authentication for clients: require an API key to access the proxy, separate from the provider API keys. (6) Logging: log metadata (model, tokens, latency) but NOT prompt/response content. Please provide the complete config.yaml and the docker run command to start the proxy.

PROMPT: Advanced Routing and Load Balancing

I have a LiteLLM proxy serving 3 agent platforms (Agent-S, Browser Use, OpenHands) and I need to implement advanced routing strategies. Please provide: (1) Load balancing: how to distribute requests across multiple API keys for the same provider (e.g., 3 OpenAI API keys with 500 RPM each, giving 1500 RPM total). Show the config.yaml setup. (2) Priority routing: how to ensure Agent-S requests (desktop automation, latency-sensitive) get priority over OpenHands batch tasks. (3) Model routing by use case: how to automatically route image-heavy requests (Agent-S screenshots) to models with vision capability, and text-only requests to cheaper models. (4) Cooldown strategy: if a provider returns consistent errors, how to temporarily remove it from the rotation and re-add it after a cooldown period. (5) Canary deployment: how to route 10% of requests to a new model version (e.g., GPT-4o vs. GPT-4o-mini) for A/B quality comparison without affecting the remaining 90% of traffic.

PROMPT: Monitoring, Alerting, and Cost Control

I need to monitor my LiteLLM proxy in production to prevent cost overruns and detect anomalies. Please provide: (1) How to expose LiteLLM metrics to Prometheus: what endpoints, what metric names, and what labels are available. (2) Alert rules for: daily spend exceeding $50, error rate exceeding 10% over 5 minutes, a single API key consuming more than 50% of the rate limit, and a single model latency p99 exceeding 30 seconds. (3) A dashboard (Grafana JSON or description) showing: real-time request rate by model, cumulative spend by day, error rate by provider, and latency distribution by model. (4) Cost optimization strategies: how to identify which agent platform or task type is consuming the most tokens, and how to implement per-project or per-user cost allocation. (5) Automated cost controls: how to set a hard monthly spend limit that stops all proxy requests when exceeded, and a soft limit that sends a warning at 80% of budget.

8.5 Integration and Architecture Prompts

PROMPT: Orchestrator Design for Composable Stack

I am building an orchestrator that coordinates three AI agent platforms: Agent-S (desktop automation), Browser Use (web automation), and OpenHands (software development), all routing LLM calls through LiteLLM. Please provide: (1) A recommended architecture for the orchestrator: should it be a Python

FastAPI service, a Node.js server, or something else? Consider that it needs to manage long-running agent sessions (5-60 minutes), handle WebSocket connections for real-time status, and persist state in a database. (2) The task dispatch algorithm: given a user request (e.g., "automate my daily report generation"), how should the orchestrator decompose it into subtasks and assign each to the appropriate platform? What heuristics should it use (e.g., web-related tasks to Browser Use, code generation to OpenHands, desktop interaction to Agent-S)? (3) The state management model: how to track the state of each subtask (pending, running, completed, failed) and handle dependencies between subtasks (e.g., OpenHands must finish writing code before Browser Use can test it). (4) Error handling: if a subtask fails, should the orchestrator retry it, reassign it to a different platform, or escalate to a human? Design a decision tree for failure handling. (5) The API contract between the orchestrator and each platform: what REST endpoints or message queue topics does each platform expose for task submission, status queries, and result retrieval?

PROMPT: End-to-End Workflow Implementation

I need a concrete implementation of an end-to-end workflow that exercises all three agent platforms (Agent-S, Browser Use, OpenHands) coordinated by an orchestrator and routing LLM calls through LiteLLM. The workflow is: "Monitor a competitor website for price changes, and when a change is detected, update our internal spreadsheet and notify the team via a desktop email client." Please provide: (1) A detailed step-by-step decomposition showing which platform handles each step, what data is passed between steps, and what error handling is applied. (2) Python pseudocode for the orchestrator that implements this workflow, including task submission, result collection, and inter-platform data passing. (3) The LiteLLM model selection for each step: which model alias should be used for each subtask and why. (4) How to implement the monitoring loop (Browser Use checking the website periodically) without blocking other workflows. (5) How to handle edge cases: the competitor website is down, the price change is ambiguous, the email client requires 2FA, the spreadsheet is locked by another user.

8.6 Troubleshooting and Debugging Prompts

PROMPT: Common Integration Failures

I am running a composable agent stack with Agent-S, Browser Use, OpenHands, and LiteLLM, and I am encountering the following common integration failures. Please provide root cause analysis and resolution for each: (1) Agent-S clicks the wrong UI element after a LiteLLM model switch from GPT-4o to Claude. The coordinates it targets are off by 20-50 pixels. (2) Browser Use returns "element not found" even though the element is visible on the page. This happens intermittently (30% of the time) on a specific React application. (3) OpenHands generates code that references packages not installed in the Docker sandbox. The imports fail at runtime. (4) LiteLLM proxy returns 429 (rate limited) even though I am well within the configured rate limit. The provider dashboard shows no rate limit events. (5) After running for 2 hours, the entire stack becomes unresponsive. Memory usage across all processes is normal. CPU usage is near zero. (6) A cross-platform workflow fails silently: Browser Use completes but the result is never received by Agent-S. The orchestrator shows no error logs.

PROMPT: Performance Optimization

My composable agent stack (Agent-S + Browser Use + OpenHands + LiteLLM) is functional but too slow for production workloads. A typical cross-platform workflow takes 5-8 minutes when it should take 2-3 minutes. Please provide: (1) A systematic approach to identify the bottleneck: how to measure time spent

in each component (LLM inference, action execution, DOM extraction, code compilation, inter-process communication). (2) LLM-specific optimizations: prompt compression techniques to reduce token count per step, caching strategies for repeated similar requests, and model selection guidance (when to use a fast model vs. a smart model within the same workflow). (3) Browser Use specific optimizations: how to reduce DOM extraction time on heavy pages, how to pre-warm browser instances, and how to parallelize independent browser actions. (4) Agent-S specific optimizations: how to reduce screenshot processing time, how to use region-of-interest capture instead of full desktop screenshots, and how to batch multiple actions into a single LLM call. (5) OpenHands specific optimizations: how to reduce sandbox startup time, how to cache compiled dependencies, and how to stream test results instead of waiting for the full suite to complete.

9. Quick Reference

9.1 Stack Component Summary

Component	Role	License	Stars	Language	Key Feature
Agent-S	Desktop Automation	Apache-2.0	11,391	Python	Screenshot-based visual grounding
Browser Use	Web Automation	MIT	94,482	Python	DOM extraction + 15 LLM providers
OpenHands	Dev Platform	MIT	74,000	Python/TS	Sandboxed code execution + web UI
LiteLLM	LLM Router	MIT	16,000+	Python	Unified API for 100+ providers

9.2 Critical Configuration Endpoints

Platform	Config Method	LLM Endpoint Variable	Default Port
Agent-S	YAML + env vars	OPENAI_BASE_URL or LITELLM_URL	N/A (client)
Browser Use	Python config	LLM_BASE_URL	N/A (client)
OpenHands	Docker env vars	LLM_BASE_URL	3000 (UI) / 8000 (API)
LiteLLM	config.yaml	N/A (it IS the endpoint)	4000 (proxy)

9.3 Verification Priority Matrix

When time is limited, prioritize verification in the following order. Items at the top have the highest impact on production stability and the highest risk of failure.

Priority	Verification Area	Risk if Skipped	Estimated Time
1 (Critical)	LiteLLM fallback routing	All agents fail on provider outage	2 hours
2 (Critical)	Security audit (sandbox escape)	Host system compromise	4 hours
3 (High)	End-to-end integration workflow	Stack does not work as a whole	4 hours
4 (High)	Rate limiting and cost controls	Unlimited spend, provider bans	2 hours
5 (High)	LLM provider switching (zero code change)	Locked to single provider	2 hours
6 (Medium)	Long-running session memory test	Memory leak crashes in production	3 hours
7 (Medium)	Concurrent agent operation	Race conditions, data corruption	3 hours
8 (Medium)	License compliance scan	Legal risk from copyleft contamination	1 hour
9 (Low)	Performance benchmarking	Suboptimal but functional	4 hours
10 (Low)	Documentation and runbooks	Slower incident response	4 hours

Next Steps: Begin with Priority 1 (LiteLLM fallback routing) and work down the matrix. Each verification area produces a pass/fail result with evidence. Only proceed to deployment when all Critical and High items pass. Medium and Low items can be addressed in the first production iteration with documented known issues.